Orbis: Guiding Symbolic Execution Techniques to Maximize Option-Related Branch Coverage

Minjong Kim
Sungkyunkwan University
Suwon, Republic of Korea
minjong.kim@skku.edu

Sungjae Hwang
Sungkyunkwan University
Suwon, Republic of Korea
sungjaeh@skku.edu

Sooyoung Cha*
Sungkyunkwan University
Suwon, Republic of Korea
sooyoung.cha@skku.edu

Abstract

We present Orbis, a complementary approach that guides symbolic execution techniques toward maximizing option-related branch (OB) coverage. Although state-of-the-art symbolic execution techniques have substantially improved code coverage and bug detection, existing techniques often fail to exercise core program functionalities, which are frequently exposed as program options. To quantify this limitation, we evaluate how effectively current techniques cover option-related branches and find that they reach only 8.0% of the targeted OBs, which are accessible only through specific options. Orbis overcomes this challenge by systematically guiding symbolic execution to improve OB coverage. For a given program, Orbis automatically identifies all defined options and determines their associated branches by analyzing dynamic execution traces. It then iteratively constructs target option arguments by stochastically combining identified options according to their estimated potential, directs symbolic execution to explore states influenced by these arguments, and refines their potential while simultaneously reducing SMT-solver invocations using data accumulated during symbolic execution. Experimental results on 15 open-source C programs show that integrating Orbis with four independent state-of-the-art techniques significantly increases OB coverage to 323.0%, boosts total branch coverage by 112.1%, and detects five additional unique bugs on average compared to the standalone techniques.

CCS Concepts

• **Software and its engineering** → **Software testing and debugging**; • **Computing methodologies** → *Machine learning*.

Keywords

Symbolic execution, Program-option based testing, Software testing

ACM Reference Format:
Minjong Kim, Sungjae Hwang, and Sooyoung Cha. 2026. Orbis: Guiding Symbolic Execution Techniques to Maximize Option-Related Branch Coverage. In *Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering (ASE '26), October 12–16, 2026, Munich, Germany.* ACM, New York, NY, USA, 13 pages. https://doi.org/10.1145/3832783.3837526

*Corresponding author.

ASE '26, Munich, Germany

ACM ISBN 979-8-4007-2882-2/2026/10
https://doi.org/10.1145/3832783.3837526

1 Introduction

Symbolic execution [5, 6, 20, 38] is a sophisticated software testing method that automatically generates test-cases, achieving high code coverage of the target program and uncovering diverse hidden bugs. To generate such test-cases, this method maps program inputs into symbolic variables and systematically explores each program path, which consists of exercised symbolic branch conditions containing these variables. Specifically, symbolic execution follows an iterative process that selects, executes, and forks a "state" from the total set of states, which are continuously updated during testing. In particular, due to this fork process, which splits a state into two separate states when both sides of a symbolic branch condition are feasible using the SMT solver [15, 17, 37], symbolic execution faces two notorious challenges: the state-explosion problem [1, 3, 10, 22, 43] and the high cost of SMT solving [24, 30, 34, 39, 42].

Although state-of-the-art techniques [8, 10, 22, 39, 41, 42] aim to address these challenges of symbolic execution and have remarkably enhanced its performance, we observed that these recent techniques still fail to effectively execute the core functionalities of the program. Since a program's core functionalities are typically implemented as program options, we evaluate how effectively these techniques cover branches related to program options. In this work, we newly define option-related branches (OBs) as branches associated with specific options, meaning they can be reached by utilizing these options. However, as demonstrated in Section 2, four independent state-of-the-art techniques for symbolic execution—search heuristic [22], state-pruning [10], SMT solving cost reduction [42], and external parameter tuning [8]—exhibited unsatisfactory performance. Despite their advancements, these techniques covered only 7.9% of the targeted total OBs across open-source C programs, leaving a significant portion of program functionalities untested.

In this paper, we present Orbis, a new complementary approach for existing symbolic execution techniques that guides their exploration toward maximizing OB coverage without prior knowledge. By doing so, Orbis improves both total branch coverage and bug-finding capability. First, Orbis automatically extracts useful data (e.g., options, option-related branches) by comparing dynamic execution traces with and without each option. It then iteratively performs three stages: (1) Construct, which builds a target option argument by estimating the potential of each option and stochastically combining them; (2) Guide, which sets the technique's initial states to steer exploration toward branches influenced by the constructed argument; and (3) Update, which refines the argument's potential based on generated test-cases and feeds back more promising arguments and states into symbolic execution. In addition, Orbis leverages this updated information to reduce SMT-solver calls, further enhancing efficiency.

Algorithm 1 Symbolic Execution

Input: Program (*pgm*), budget (*time*), arguments (*Arg*), and promising states (S_P).
Output: Covered branches (B) and test-cases (T)

```
1:  procedure Execute(pgm, time, Arg, S_P)
2:      if S_P = ∅ then S ← {s_0} else S ← S_P        ▷ s_0 = (stmt_0, M_0, true)
3:      T ← ∅
4:      repeat
5:          s ← Choose(S)                              ▷ s = (stmt, M, Φ)
6:          if stmt is an if/else statement with a condition φ then
7:              if Sat(Φ ∧ φ) then S ← S ∪ {(stmt_1, M', Φ ∧ φ)}
8:              if Sat(Φ ∧ ¬φ) then S ← S ∪ {(stmt_2, M', Φ ∧ ¬φ)}
9:          else if stmt is a halt statement then
10:             Arg' = Solve(Φ)
11:             T ← T ∪ {t}                            ▷ t = Arg · Arg'
12:         else
13:             S ← S ∪ {s'}                           ▷ s' = (stmt', M', Φ)
14:     until time expires (or S = ∅)
15:     B ← Run(pgm, T)
16:     return B, T
```

Experimental results demonstrate that Orbis effectively guides four distinct state-of-the-art techniques for symbolic execution toward maximizing OB coverage: search strategy (Learch) [22], state-pruning strategy (Homi) [10], SMT-solving strategy (AAQC) [42], and parameter-tuning strategy (SymTuner) [8]. We integrated Orbis with each technique and evaluated its effectiveness on 15 open-source C programs ranging from 4K to 48K branches. In terms of coverage, the Orbis-guided techniques covered 323.0% more option-related branches, leading to a 112.1% increase in total branch coverage compared to their standalone versions. For bug detection, Orbis enabled the pure techniques to discover an average of five additional unique bugs while ensuring that none detected by the pure techniques were missed. We further evaluated Orbis by comparing it with three state-of-the-art fuzzers specialized in testing program options [26, 44, 45], and assessed its generality by integrating it with concolic testing [9, 13], a widely used variant of dynamic symbolic execution.

Contributions. We summarize our contributions below:

- **Underexplored problem**: We show that state-of-the-art symbolic execution techniques do not effectively cover option-related branches, motivating the need for techniques that systematically guide symbolic execution toward option-related branches.
- **New technique**: We introduce Orbis, a novel technique that enables diverse symbolic execution methods to maximize OB coverage. Our core contribution is a specialized algorithm that iteratively constructs target option arguments, sets the symbolic executor's initial states to focus on these arguments, and refines their potential while simultaneously reducing SMT-solving invocations based on data accumulated during testing.
- **Extensive evaluation**: We demonstrate the effectiveness of Orbis by integrating it with four distinct symbolic execution techniques and comparing their performance with the pure techniques across 15 open-source C programs. We make Orbis publicly available: https://doi.org/10.5281/zenodo.21767392.

2 Preliminaries

2.1 Symbolic Execution

Symbolic execution [5] systematically generates test-cases likely to increase code coverage by iteratively updating so-called "states". We define a single state as a tuple of three elements: (1) the next statement to execute (*stmt*), (2) a symbolic memory (M) that maps each concrete variable to a symbolic variable, and (3) a path condition (Φ) capturing a sequence of exercised symbolic branch conditions.

Algorithm 1 details the symbolic execution process. It accepts four inputs: the target program (*pgm*) to test, the time budget (*time*), a set of concrete arguments (*Arg*), and a set of promising states (S_P). For now, we set aside the last two inputs, *Arg* and S_P, which will be explained later; general symbolic execution techniques [5, 7, 10, 22] typically test the target program without taking *Arg* and S_P as input (i.e., $Arg{=}\emptyset$, $S_P{=}\emptyset$). At line 2, the algorithm first initializes the set S of states based on S_P. If S_P is empty, S is initialized as a singleton set containing a state $(stmt_0, M_0, true)$. Otherwise, Algorithm 1 replaces S with the set S_P of promising states. At line 3, the set of test-cases (T) is initialized as an empty set.

After initialization, the algorithm repeatedly selects, forks, and updates a state in the set S until the time budget is exhausted, thereby accumulating test-cases T (lines 4–14). At line 5, the Choose function selects a state from S, and the selected state s is removed from S (i.e., $S \leftarrow S \setminus \{s\}$). If the next statement of the selected state, $(stmt, M, \Phi)$, is an if/else statement with condition φ, Algorithm 1 evaluates the satisfiability of the updated path-conditions, $(\Phi \wedge \varphi)$ and $(\Phi \wedge \neg\varphi)$, respectively. If both conditions are feasible, the algorithm forks the state s into two separate states: $(stmt_1, M', \Phi \wedge \varphi)$ and $(stmt_2, M', \Phi \wedge \neg\varphi)$ at lines 7–8.

If the next statement of the state $(stmt, M, \Phi)$ is a halt statement, Algorithm 1 invokes the SMT solver (the Solve function) at line 10 to solve the accumulated path-condition Φ and obtain a satisfiable model, denoted as the symbolic input Arg'. At line 11, it then generates a test-case t by concatenating the given concrete argument *Arg* with Arg'. For example, if the concrete argument (*Arg*) is "-bc" and the SMT solver returns (Arg' = "-R") as a model of Φ, the resulting test case (t) is "-bc -R". Lastly, if the next statement of the state s is another type of statement (e.g., load, store), the algorithm updates s into s' accordingly. Once the time budget expires, the Run function runs the target program (*pgm*) with all generated test-cases in T and calculates the set B of branches covered by T. Finally, Algorithm 1 returns the set of generated test-cases and their branch coverage.

Recent advancements in symbolic execution have significantly improved its effectiveness based on Algorithm 1. For instance, search strategies [22, 41], corresponding to the Choose function at line 5, prioritize selecting the most promising states first from the set S based on predefined criteria. Conversely, state-pruning strategies [10, 11] aim to eliminate redundant states from S by terminating them early. Some techniques—such as the Solve function at line 10—focus on efficiently reducing constraint solving costs [39, 42]. Additionally, an orthogonal technique [8] attempts to automatically tune external parameters of the symbolic execution engine (the Execute procedure). These techniques significantly improve code coverage and facilitate the discovery of subtle bugs, compared to the generic symbolic execution (Algorithm 1).

2.2 Limitation of Symbolic Execution

Our key observation is that recent techniques [8, 10, 22, 42] for symbolic execution still struggle to effectively test the core functionalities of the target program, despite their remarkable enhancement. Specifically, since these core functionalities are typically implemented as program options, we evaluate how effectively these

Table 1: Symbolic execution on option-related code regions.

Program	KLEE [5]		Homi [10]		Learch [22]		AAQC [42]		SymTuner [8]	
	Opt	OB	Opt	OB	Opt	OB	Opt	OB	Opt	OB
xorriso	3.2%	0.8%	3.8%	0.9%	3.2%	1.0%	3.8%	0.8%	3.8%	1.1%
sqlite	21.2%	1.2%	63.6%	1.3%	57.6%	1.4%	3.0%	0.1%	60.6%	1.8%
gcal	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%	11.1%	16.0%
patch	27.7%	25.7%	30.8%	24.4%	7.7%	13.9%	10.8%	23.6%	49.2%	34.2%
csplit	26.7%	13.3%	26.7%	17.3%	13.3%	12.7%	13.3%	10.7%	26.7%	18.0%
ls	18.1%	4.5%	14.5%	3.7%	43.4%	2.2%	10.8%	1.3%	53.0%	5.8%

```
1  int main (int argc, char **argv) {
2      int c; char *format = NULL; char *style = NULL;
3      while (true) {
4          c = getopt(argc, argv, "l")
5          if (c == "l")
6              format = "long";
7          else
8              break;
9      }
10     if (strcmp(format, "long") == 0)
11         style = "time_style";
12 }
```

Figure 1: Example of option-related branches.

state-of-the-art techniques cover branches related to program options. In this work, we newly define option-related branches (OBs) as branches associated with a specific option and reachable by utilizing that option. More precisely, an OB is a conditional branch whose condition contains a (variable, value) pair that is newly defined or updated only when the corresponding option is enabled.

Figure 1 illustrates an example of OBs using code snippets from the `ls` program. Suppose we execute `ls` with the option "-l" (e.g., ./ls -l). In this case, the variable "c" is assigned the value "l", which is then used as a branch condition in the if statement at line 5. We classify this branch as an OB. Furthermore, since the "format" variable is set to "long" in this execution, the true branch of the if statement at line 10 becomes reachable, and we also classify this branch as an OB.

In terms of OBs, state-of-the-art techniques were largely ineffective across open-source C programs. Table 1 shows that KLEE [5], a widely-used symbolic executor, and four distinct state-of-the-art techniques covered only 7.9% of the targeted total option-related branches on average across the six programs. Specifically, these techniques include search strategy (Learch [22]), constraint-solving reduction strategy (AAQC [42]), state-pruning strategy (Homi [10]), and parameter tuning strategy (SymTuner [8]) for symbolic execution. Surprisingly, all symbolic executors except SymTuner even failed to generate test cases containing a single valid option for the target program (`gcal`). The fundamental reason is that `gcal` defines particularly long options, with an average extracted-option length of 22 characters. Longer option arguments require solving more complex path conditions, substantially increasing SMT-solving overhead. Consequently, existing symbolic execution techniques struggle to generate even a single valid option within the given testing budget. The column "Opt" represents the proportion of program options exercised at least once by the test-cases generated by each technique. Within a 24-hour budget, they included only 20.2% of the valid options defined across the six programs. Table 1 also highlights that increasing OBs is non-trivial; merely using each program option is insufficient. For `sqlite`, the five techniques exercised valid options relatively well (41.2% on average), yet achieved only 1.2% OB coverage.

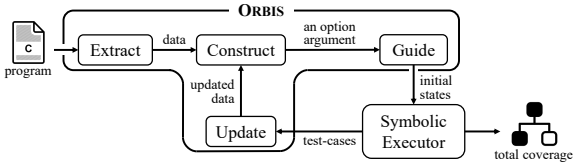

Figure 2: The overview of Orbis.

These limitations motivate our complementary approach, which guides symbolic execution techniques towards maximizing *OB* coverage from scratch, thereby improving total branch coverage and bug-finding capability. This is feasible because OB coverage and total branch coverage are strongly correlated in our benchmark suite, with an average Pearson coefficient of 0.902, as demonstrated in Section 4.2. Our work thus tackles two key challenges: (1) automatically extracting OBs without prior knowledge, and (2) effectively guiding symbolic execution techniques toward these branches.

3 Our Approach

3.1 Overview

Figure 2 illustrates the interaction between Orbis and the symbolic executor. Given a target program, Orbis coordinates symbolic execution through four stages: Extract, Construct, Guide, and Update. In the Extract stage, Orbis gathers useful data (i.e., options, option-related branches) by comparing dynamic execution traces with and without each option. This information is then used to guide the symbolic executor toward maximizing OB coverage.

Using the extracted data, Orbis iteratively performs the remaining three stages while interacting with the symbolic executor. In the Construct stage, Orbis generates a target option argument by estimating the potential of each option and stochastically combining options, which the symbolic executor will then focus on. The Guide stage initializes the symbolic executor's states through seeding, enabling effective exploration of option-related branches near the formed option argument. After symbolic execution runs using both the option argument and the initialized states, the Update stage analyzes the generated test-cases and refines the data used in the Construct and Guide stages. Orbis also leverages this updated data to reduce SMT-solver invocations, further improving efficiency. This process repeats until the entire budget is exhausted. Finally, Orbis returns the total set of covered branches across all iterations.

3.2 Orbis

Algorithm 2 details how Orbis guides the symbolic executor to increase OB coverage from scratch. The algorithm takes as input the target program (*pgm*) and the total testing budget (*time*), and returns the total set of covered branches (*TotalB*) once the budget expires. Before the main stages, Algorithm 2 initializes two sets—the test-case set *TotalT* and the covered branch set *TotalB*—as empty. It also divides the total budget (*time*) into smaller time segments, referred to as $time_s$, to enable multiple interactions with the symbolic executor (Algorithm 1). These repeated interactions allow Orbis to iteratively refine and provide increasingly effective option arguments and their associated states to the symbolic executor, thereby enhancing OB coverage. In our experiments, we set $time_s$ to 120 seconds. At line 15, Orbis doubles $time_s$ because repeated

Algorithm 2 Orbis

Input: Program (*pgm*) and budget (*time*)
Output: A total set of covered branches (*TotalB*)
1: $\langle TotalT, TotalB, time_s \rangle \leftarrow \langle \emptyset, \emptyset, 120s \rangle$
2: /* Step 1: Extract Stage */
3: $D \leftarrow \text{Extract}(pgm)$ ▷ $(o, OB) \in D$
4: $ArgD \leftarrow \{(\{o\}, OB, \emptyset) \mid (o, OB) \in D\}$
5: **repeat**
6: **for** $i = 1$ **to** $|D|$ **do**
7: /* Step 2: Construct Stage */
8: $Arg_{new} \leftarrow \text{Construct}(ArgD)$ ▷ $o \in Arg_{new}$
9: /* Step 3: Guide Stage */
10: $S_P \leftarrow \text{Guide}(pgm, Arg_{new}, TotalT)$
11: $B, T \leftarrow \text{Execute}(pgm, time_s, Arg_{new}, S_P)$
12: $\langle TotalB, TotalT \rangle \leftarrow \langle TotalB \cup B, TotalT \cup T \rangle$
13: /* Step 4: Update Stage */
14: $ArgD \leftarrow \text{Update}(ArgD, Arg_{new}, B)$
15: $time_s \leftarrow time_s \times 2$
16: **until** *time* expires
17: **return** *TotalB*

interactions progressively increase confidence in the constructed option arguments and their associated symbolic states. If the doubled value of $time_s$ exceeds the remaining testing budget, Orbis sets $time_s$ to the remaining budget before invoking the symbolic executor. Consequently, the cumulative execution time never exceeds the total testing budget (*time*).

Extract. In the Extract stage, Algorithm 2 (Orbis) collects the data *D* required to guide the symbolic executor (line 3). Each element of *D* is defined as a pair (*o*, *OB*), where *o* is an option defined in the target program and *OB* is the set of its corresponding option-related branches. This stage constructs *D* by extracting data for individual options, such that each element corresponds to a single option (e.g., "-b", "-c") and its option-related branches, rather than to combined options (e.g., "-bc"). In addition, for options that require parameters, Orbis identifies valid parameter values from the program's help output. For example, if the help output indicates that the "--sort" option accepts parameter values such as {time, size}, Orbis extracts concrete option arguments such as "--sort=time" and "--sort=size" and treats each option–value pair as an independent option argument. Consequently, the dependency between an option and its required parameter is naturally handled by representing them as a single concrete option argument. To build *D*, Orbis first exhaustively identifies all available options in the program's help script. For example, the help output of the `ls` program yields candidate options, such as "–l" and "--sort=time":

```
-l            use a long listing format
--sort=WORD   sort by WORD instead of name: time(-t), size(-S)
```

We acknowledge that some hidden options may not appear in the help script. However, as discussed in Section 4.8, these constitute only 0.9% of all options across our benchmark suite, and even if extracted, their impact on improving symbolic execution performance is negligible. Therefore, our approach adopts the cost-effective strategy of extracting options solely from the help script.

Once valid options are extracted, for each option, Orbis automatically (1) generates two concrete inputs: one that includes the option (e.g., ./ls -l) and one that excludes it (e.g., ./ls); (2) compares the execution traces obtained from concretely executing these two inputs; and (3) collects all variables directly associated with that option. Specifically, Orbis considers variables that are newly defined or updated only when the input containing the option is executed. To achieve this, Orbis employs the Trace function, which takes a command (*cmd*) as input and collects trace information (*Traces*) through concrete execution. Each element of *Traces* is a pair (*var*, *Val*), where *var* is a variable that is defined or updated during execution, and *Val* is the set of values assigned to that variable:

$$(var, Val) \in \text{Trace}(cmd) = Traces$$

For example, consider the code snippet in Figure 1. If the command with the `-l` option is "`./ls -l`", then the trace is as:

$$Traces = \begin{Bmatrix} (\text{format}, \{\text{NULL}\}), (\text{style}, \{\text{NULL}\}), (\text{c}, \{\text{“l”}, \text{-1}\}), \\ (\text{format}, \{\text{“long”}\}), (\text{style}, \{\text{“time\_style”}\}) \end{Bmatrix}$$

In this execution, the variable "c" is first assigned "l" when the loop begins, and later assigned `-1` when the loop exits. Thus, the values of "c" are represented as the set {"l", -1}. Similarly, by tracing the base command "`./ls`" without options, Orbis obtains another trace (*Traces*′):

$$Traces' = \{(\text{format}, \{\text{NULL}\}), (\text{style}, \{\text{NULL}\}), (\text{c}, \{\text{-1}\})\}$$

Now, Orbis collects the unique trace information for each option by computing the difference between *Traces* and *Traces*′, which corresponds to the elements observed exclusively when the option is included in the command. This approach effectively avoids false positives because the compared commands are identical except for the option under analysis. For example, the unique trace associated with the "–l" option of the `ls` program is as:

$$\{(\text{c}, \text{“l”}), (\text{format}, \text{“long”}), (\text{style}, \text{“time\_style”})\}$$

Using the unique trace for each option, Orbis inspects the target program's source code and adds all conditional branches that reference any element in the trace to that option's set of option-related branches. This process produces the data *D*, where each element is a pair (*o*, *OB*) of an option *o* and its corresponding branches *OB*. With this method, Orbis precisely extracts the true branches at lines 5 and 10 in Figure 1 as those associated with the "-l" option. This trace-difference process captures dependencies that arise when an option-specific value propagates through variable assignments and is later used in a conditional branch. For instance, in Figure 1, the value "long" assigned to the "format" variable is later referenced at line 10, so this branch is identified as an OB of the "-l" option.

With the extracted data *D*, Algorithm 2 initializes the set *ArgD* at line 4, which will be utilized in the next step:

$$(Arg, OB, CovOB) \in ArgD, \text{ where } B \in CovOB$$

Here, the first element in *ArgD* represents an option argument (*Arg*), which consists of the set of program options. *OB* denotes the total set of option-related branches corresponding to *Arg*, while *CovOB* represents the set of option-related branches covered during each iteration between lines 5–18. For example, an element in *ArgD* can be defined as $(Arg, \{b_1, b_2, b_3, b_4\}, \{\{b_1\}, \{b_1, b_2\}\})$. This indicates that the option argument *Arg* contains a total of four option-related branches, ranging from b_1 to b_4, while covering b_1 in the first iteration and b_1 and b_2 in the second iteration. We deliberately define *CovOB* as a set of sets to facilitate scoring the impact of each option argument in the next Construct step.

Construct. The goal of the Construct stage is to form a new option argument (Arg_{new}) from the set *ArgD* to guide the symbolic

executor's state exploration by probabilistically combining arguments in *ArgD* based on their effectiveness (line 8). The newly generated argument is then passed to the symbolic executor (line 11). Recall that the symbolic executor (Algorithm 1) utilizes an option argument to generate each test-case by concatenating the option argument and symbolically generated arguments, which are produced by the SMT solver. This approach effectively forces the symbolic executor to explore code regions related to the option argument.

To construct a new option argument (Arg_{new}), the Construct function computes a score for each option argument (*Arg*) in *ArgD* and then stochastically forms a new option argument based on their scores. Specifically, to assign a score to each option argument, we transform it into a feature vector. In our approach, we define three atomic features that estimate the effectiveness of an option argument in terms of OB coverage while keeping feature extraction lightweight. A feature (feat) is a function that takes an option argument as input and returns a real value: feat : $\mathbb{O} \rightarrow \mathbb{R}$, where $\mathbb{O}$ denotes the set of all possible option arguments. That is, each *Arg* is transformed as: $\langle \text{feat}_1(Arg), \text{feat}_2(Arg), \text{feat}_3(Arg) \rangle$.

Finally, we normalize each value in the feature vector using Min-Max normalization and then compute the score of *Arg* by summing up all three normalized values. The three predefined features are described in detail below:

(1) **uncovered OB (**feat_1**)**: This feature computes the ratio of option-related branches that are not yet covered within the total set *OB* corresponding to *Arg*, defined as:

$$\text{feat}_1(Arg) = \frac{|OB \setminus \bigcup_{B \in CovOB} B|}{|OB|}, \text{ where } (Arg, OB, CovOB) \in ArgD$$

Intuitively, this feature estimates the potential of *Arg* based on how many of its associated branches remain uncovered.

(2) **number of attempts (**feat_2**)**: This feature assigns higher scores to arguments that have been selected less frequently in previous iterations, thereby promoting diversity in the selection of option arguments. It is calculated as:

$$\text{feat}_2(Arg) = \frac{1}{|CovOB|}, \text{ where } (Arg, \_, CovOB) \in ArgD$$

(3) **average OB (**feat_3**)**: This feature favors arguments that, on average, have covered more option-related branches in previous iterations, thereby enhancing the efficiency of newly constructed arguments. It is computed as:

$$\text{feat}_3(Arg) = \sum_{B \in CovOB} \frac{|B|}{|CovOB|}, \text{ where } (Arg, \_, CovOB) \in ArgD$$

After scoring each option argument in *ArgD*, Orbis assigns a sampling probability to each argument proportional to its score. Orbis then samples the option arguments from *ArgD* according to these probabilities and constructs a new option argument as the union of the selected arguments. For example, consider three individual option arguments in *ArgD*: "{-R}", "{-n}", "{-w}", with probabilities of 27.3%, 65.5%, and 7.2%, respectively. If the first two arguments are sampled according to their probabilities, the resulting combined option argument becomes "{-Rn}". Orbis adds this newly constructed argument to *ArgD*, enabling increasingly complex combined option arguments as it interacts with the symbol executor. Empirically, across our benchmark suite, Orbis generated option arguments combining 4 individual options on average (up to 13.2 options). Through this iterative construction process, Orbis implicitly favors beneficial combinations of inter-dependent options. Option combinations that work well together tend to cover more option-related branches, receive higher scores, and are therefore sampled more frequently in subsequent iterations. In contrast, combinations that interact poorly typically contribute little additional coverage, receive lower scores, and are consequently selected less often. Finally, at line 11, Algorithm 2 (Orbis) delivers the newly constructed option argument (Arg_{new}) to the symbolic executor.

Guide. The Guide stage initializes the symbolic executor's initial states (Algorithm 1) to steer exploration toward option-related branches associated with the constructed option argument (Arg_{new}) at line 10. We achieve this through seeding. Given seed inputs, seeding efficiently initializes symbolic executor states without requiring expensive satisfiability checks during seed regeneration. Thus, selecting seed inputs related to Arg_{new} provides an effective starting point for exploring its associated branches.

We first explain how the Seeding procedure operates on a given seed input. Let SatPC(*seed*) denote the set of satisfiable path conditions encountered during symbolic execution of the program using the given seed input: SatPC(*seed*) $= \{\Phi_1, \Phi_2, \cdots, \Phi_n\}$. To construct SatPC(*seed*), symbolic inputs are concretized using the seed values, and each path condition that evaluates to true under the seed assignment is collected without invoking the SMT solver. For example, consider an execution path with the branch conditions $x > 10$ and $x = 20$. Given a seed assigning $x = 20$, the satisfiable path condition $x > 10 \wedge x = 20$ evaluates to $20 > 10 \wedge 20 = 20$ and is therefore included in SatPC(*seed*). Using this SatPC function, the Seeding procedure is defined as:

```
1:  procedure Seeding(seed)
2:      S ← {s_0}                                   ▷ s_0 = (stmt_0, M_0, true)
3:      repeat
4:          s ← Choose(S)                           ▷ s = (stmt, M, Φ)
5:          if stmt is an if/else statement with a condition φ then
6:              if ((Φ ∧ φ) ∈ SatPC(seed)) then S ← S ∪ {(stmt_1, M', Φ ∧ φ)}
7:              if ((Φ ∧ ¬φ) ∈ SatPC(seed)) then S ← S ∪ {(stmt_2, M', Φ ∧ ¬φ)}
8:          else
9:              S ← S ∪ {s'}                        ▷ s' = (stmt', M', Φ)
10:     until stmt is a halt statement
11:     return S
```

This procedure is similar to Algorithm 1 but avoids SMT solving during state forking. At lines 6–7, the state *s* is forked into two new states without solver calls if the satisfiability of $(\Phi \wedge \varphi)$ and $(\Phi \wedge \neg\varphi)$ has already been confirmed in SatPC(*seed*). This procedure repeats until the next statement of the state reaches a halt statement, at which point it returns the set of states *S* (line 11). Consequently, the Guide stage builds promising states S_P by running the Seeding procedure with each seed in the carefully selected seed set (*SD*) and accumulating the resulting states as: $S_P = \bigcup_{seed \in SD} \text{Seeding}(seed)$.

To initialize these promising states, Algorithm 2 first splits the given option argument (Arg_{new}) into individual options (e.g., "-Rn" $\rightarrow$ {-R, -n}). For each divided option, it then selects a promising test-case from those generated during the repeated symbolic execution runs (lines 5–18). In this work, a test-case is considered promising if it achieves the highest OB coverage while containing the divided option. The selected test-cases, together with Arg_{new}, form the seed input set as: $SD = \{Arg_{new}\} \cup \{\text{Select}(o, T_o) \mid o \in Arg_{new}\}$, where T_o denotes the set of test-cases containing each option *o* among all the generated test-cases: $T_o = \{t \mid o \in t \wedge t \in TotalT\}$. Specifically,

given the union of all option-related branches:

$$UnionOB = \bigcup_{(\_,OB)\in D} OB$$

the Select function chooses from T_o the test-case whose execution covers the largest number of branches in *UnionOB*:

$$\underset{t\in T_o}{\text{argmax}} |B \cap UnionOB|, \text{ where } B = \textsc{Run}(pgm, \{t\}).$$

By applying the Select function to each divided option, Algorithm 2 constructs the seed input set *SD* and ultimately derives the promising states S_P from it (line 10). Empirically, this procedure enables Orbis to derive effective initial states from the delicately selected seeds within only 9.6% of the total time budget on average, resulting in a 21.5% branch coverage improvement, as shown in Section 4.4.

Update. The Update stage in Algorithm 2 improves the efficiency of the iterative Construct and Guide stages by updating the *ArgD* set, which is used for scoring each option argument (line 14). Recall that each element of *ArgD* consists of a triple (*Arg*, *OB*, *CovOB*), where *Arg* denotes an option argument, *OB* represents the total set of option-related branches associated with *Arg*, and *CovOB* indicates the set of option-related branches covered during each iteration (lines 5–16). When a new option argument Arg_{new} is constructed, Algorithm 2 updates *AllArgs* as follows. First, it collects all option arguments currently in *ArgD* as:

$$AllArgs = \{Arg \mid (Arg, \_, CovOB) \in ArgD\}$$

If Arg_{new} belongs to *AllArgs*, the algorithm updates the existing entry by extending its coverage set:

$$(Arg_{new}, OB, CovOB \cup \{CovOB_{new}\}) \in ArgD, CovOB_{new} = OB \cap B,$$

where the intersection between *OB* and *B* denotes the covered option-related branches set associated with Arg_{new}. Conversely, if Arg_{new} does not exist in *ArgD*, Orbis adds a new element for Arg_{new}:

$$ArgD = ArgD \cup \{(Arg_{new}, OB_{\text{Arg}}, \{OB \cap B\})\}$$

where OB_{Arg} denotes the union of option-related branches corresponding to each option in Arg_{new} as:

$$OB_{\text{Arg}} = \{b \in OB \mid (o \in Arg_{new}) \wedge (\{o\}, OB, \_) \in ArgD\}$$

Through this update, *ArgD* maintains accurate OB coverage information, enabling Algorithm 2 to iteratively refine option arguments and the corresponding symbolic executor states. The process repeats until the time budget is exhausted, gradually converging on the most effective option arguments and promising states.

Furthermore, Orbis leverages *ArgD* to reduce SMT solver invocations during symbolic execution. Unlike standard symbolic execution (Algorithm 1), which invokes the solver whenever the state $(stmt, M, \Phi)$ reaches a halt statement, Orbis skips the solver call (the Solve function at line 10) under a specific condition: if there exist an option argument in *ArgD* that satisfy the path-condition Φ at the halt point. In such cases, Algorithm 1 does not call the SMT solver to generate a test-case but instead replaces Arg' at line 10 with one of those option arguments. To check whether an option argument satisfies a path-condition, Orbis substitutes the symbolic variables in Φ with the argument's concrete values and evaluates the concretized condition. If it evaluates to true, the option argument itself is generated as a test-case without SMT solving.

Suppose the path-condition Φ of a halted state and a set of candidate option arguments are:

$$\Phi = (\alpha[0]=\text{“-”}) \wedge (\alpha[1] \neq \text{“c”}),\ Args = \{\text{“-a”}, \text{“-c”}\}$$

Substituting the first candidate (“-a”) yields: (“-” = “-”) ∧ (“a” ≠ “c”), which evaluates to true. Conversely, substituting the second candidate (“-c”) yields: (“-” = “-”) ∧ (“c” ≠ “c”), which evaluates to false. In this example, Orbis does not invoke the SMT solver to find a model of Φ but instead uses the satisfied option argument (“-a”) directly as a test-case.

Note that Orbis substitutes option arguments only for option-related path conditions. Path conditions involving other symbolic inputs (e.g., file contents or directory structures) are not substituted with option arguments and are still handled by the SMT solver. Nevertheless, because path conditions derived from different symbolic input sources (e.g., option arguments and file contents) are solved independently, substituting only the option-related path conditions can still substantially reduce the overall SMT-solving overhead. As discussed in Section 4.4, this solver cost-reduction method using *ArgD* reduced the overall number of solver calls by about 16.8% across our benchmark suite.

4 Experiments

In this section, we evaluate the efficiency of Orbis by addressing the following research questions:

(1) **Branch Coverage**: To what extent does integrating Orbis with four distinct state-of-the-art techniques improve OB coverage and total branch coverage?
(2) **Bug-finding Ability**: How many unique crashes are detected by integrating Orbis?
(3) **Effectiveness of Stages**: How does each stage of Orbis contribute to performance improvement?
(4) **Generality**: Is Orbis generally applicable across different symbolic executors?

We conducted our experiments using KLEE-2.1 [5], a widely used symbolic executor. All experiments were performed on a system with two Intel Xeon Gold 6230R processors and 256GB RAM.

4.1 Experimental Settings

Baselines. To evaluate the effectiveness of Orbis's guidance, we integrated it with KLEE and four state-of-the-art techniques, comparing the integrations against their standalone versions. These techniques represent different methodological categories: search strategy (Learch) [22], state-pruning strategy (Homi) [10], constraint solving strategy (AAQC) [42], and parameter tuning strategy (SymTuner) [8]. Each enhances symbolic execution through an independent approach and has shown superior performance over its own baselines. Learch is a state selection strategy that prioritizes important states for exploration using learning-based methods. In contrast, Homi is a state elimination strategy that analyzes and excludes inefficient states, allowing the symbolic executor to focus on promising states. AAQC optimizes cache utilization to reduce SMT-solving overhead, enabling deeper state exploration. SymTuner dynamically adjusts external parameters of symbolic execution to improve performance. We evaluated the effectiveness of Orbis by comparing two settings for each baseline: (1) a pure

Table 2: 15 programs used as benchmarks.

Program	Opt	OB	B	Program	Opt	OB	B	Program	Opt	OB	B
xorriso-1.5.2	158	16,616	48,687	find-4.7.0	71	271	9,988	patch-2.7.6	65	224	6,221
sqlite-3.33.0	33	2,870	44,717	grep-3.4	81	371	8,283	objcopy-2.36	116	265	5,445
gawk-5.1.0	54	533	17,859	diff-3.7	85	1,824	7,673	ptx-8.32	35	568	5,297
gcal-4.1	126	316	15,729	make-4.3	63	96	6,893	csplit-8.32	15	30	4,491
objdump-2.36	46	335	11,975	du-8.32	44	380	6,611	ls-8.32	83	2,078	3,812

implementation without modifications, as publicly available (Pure), and (2) an integration of Orbis with each technique (Orbis).

Benchmarks. Table 2 presents the 15 open-source C programs used in our evaluation, along with the number of options in each program (Opt), the number of option-related branches targeted by Orbis (OB), and the total number of branches (B). To ensure fairness, we carefully construct our benchmark suite. We first gathered all programs used in the evaluation by the five baselines [5, 8, 10, 22, 42]. From this set, we selected 15 programs as benchmarks that, on average, covered more than 5% of the total branches across the five baselines. Programs below this threshold were excluded, as baselines primarily covered error-handling functions rather than core functionalities. Table 2 shows that the number of options in our 15 benchmarks ranges from 15 to 158 (10×), and the number of option-related branches ranges from 30 to 8,080 (269×), ensuring a diverse and representative benchmark set.

Other Settings. For all experiments, we tested each baseline with and without Orbis's guidance on each benchmark using a 24-hour budget, repeating the same experiment five times. We then reported the average results, which indicate that our experiments were conducted extensively. For instance, generating the coverage results reported in Table 3 required 18,000 hours when using a single CPU core (i.e., 24 hours × 5 trials × 15 benchmarks × 5 baselines, each tested with and without Orbis). Additionally, to ensure a fair comparison with Orbis, we evaluated baselines under both settings: (1) a single uninterrupted 24-hour run and (2) repeated shorter runs whose results were merged, matching the execution setting of Orbis. We reported the better-performing result for each baseline. The shorter budgets followed the same schedule as Algorithm 2, starting from 120 seconds and doubling after each iteration of the repeat-until loop until the total testing budget was exhausted. We also configured all baselines to potentially generate the same command-line arguments as Orbis. Specifically, each baseline was configured to generate up to three symbolic arguments, each with a maximum length equal to that of the longest extracted option (i.e., `-sym-arg max(len(options))` specified three times). Orbis used exactly the same `-sym-arg` configuration.

4.2 Branch Coverage

We evaluated the effectiveness of our approach using two metrics: option-related branch coverage, which Orbis primarily targets, and total branch coverage, a main objective of symbolic execution techniques. We obtained branch coverage results using gcov [19], a widely used code coverage measurement tool.

Option-related Branch Coverage. Table 3 shows that integrating Orbis with symbolic execution techniques significantly increases option-related branch (OB) coverage across all benchmarks compared to their pure versions. On average, applying Orbis to five symbolic execution techniques resulted in a 323.0% increase

Table 3: The option-related branch coverage achieved by baselines and their combination with Orbis.

Program	KLEE [5]		Homi [10]		Learch [22]		AAQC [42]		SymTuner [8]	
	Pure	Orbis	Pure	Orbis	Pure	Orbis	Pure	Orbis	Pure	Orbis
xorriso	131.6	**427.4**	147.8	**424.4**	160.2	**441.6**	125.2	**453.6**	179.0	**439.2**
sqlite	35.4	**86.2**	37.8	**86.8**	40.4	**86.8**	4.0	**93.6**	51.8	**91.6**
gawk	22.4	**96.4**	23.4	**87.2**	7.0	**84.2**	11.0	**90.4**	25.8	**74.0**
gcal	0.0	**181.4**	0.0	**174.0**	0.0	**186.2**	0.0	**181.2**	50.6	**169.4**
objdump	31.6	**87.4**	4.8	**86.8**	25.6	**88.8**	44.0	**84.4**	51.4	**99.0**
find	27.6	**175.0**	31.8	**171.0**	54.8	**174.8**	47.4	**166.6**	81.8	**181.8**
grep	25.2	**153.6**	28.0	**115.8**	0.0	**176.2**	28.8	**156.0**	41.4	**183.6**
diff	37.2	**150.2**	23.4	**173.6**	33.0	**174.4**	26.0	**170.4**	44.8	**162.0**
make	7.6	**65.0**	0.0	**62.0**	3.0	**67.2**	0.0	**64.2**	29.6	**64.4**
du	20.8	**68.8**	19.8	**82.0**	14.6	**86.0**	15.4	**86.4**	32.4	**94.8**
patch	57.6	**144.8**	54.6	**142.6**	31.2	**141.6**	52.8	**141.4**	76.6	**141.4**
objcopy	25.8	**204.2**	3.0	**203.8**	11.8	**212.0**	48.2	**230.6**	61.8	**216.8**
ptx	11.2	**52.2**	16.6	**52.0**	9.4	**54.0**	4.4	**52.0**	19.0	**58.0**
csplit	4.0	**20.6**	5.2	**22.8**	3.8	**22.4**	3.2	**24.4**	5.4	**26.2**
ls	94.4	**234.8**	77.6	**230.2**	46.2	**229.0**	27.6	**223.8**	121.0	**214.6**
total	532.4	**2,148.0**	473.8	**2,115.0**	441.0	**2,225.2**	438.0	**2,219.0**	872.4	**2,216.8**

in the number of covered OBs: KLEE (+303.5%), Homi (+346.4%), Learch (+404.6%), AAQC (+406.6%), and SymTuner (+154.1%). Notably, for `objcopy`, Orbis's guidance enabled Homi to cover 66.9× more OBs than Homi alone, marking the highest improvement. Surprisingly, Orbis also enables techniques that originally fail to cover any OBs to generate test-cases that effectively increase OB coverage and exercise core functionality code regions. For instance, in `gcal`, four techniques (except SymTuner) did not cover any OBs, indicating that they failed to generate a single valid option. With Orbis, however, these techniques covered an average of 180.7 OBs. A similar tendency was observed in the `make` program, where Orbis guided Homi and AAQC to achieve 7.9× more OB coverage than the average of the five techniques in their standalone versions (Pure).

The results in Table 3 were statistically significant. Across all 15 benchmark programs, the p-value between Pure and Orbis across the five baselines averaged 0.015, well below the 0.05 threshold, according to the Mann-Whitney U test [32]. In particular, for our benchmark suite, the average p-values between each of the five baselines and Orbis were all lower than 0.05: KLEE (0.011), Learch (0.010), Homi (0.009), AAQC (0.010), and SymTuner (0.011).

Total Branch Coverage. Table 4 illustrates that by targeting option-related branches, Orbis ultimately increases total branch coverage across our benchmark suite. Compared to each pure technique (Pure), Orbis achieved an average 112.1% increase in total branch coverage. Specifically, integrating Orbis led to an 101.2% increase for KLEE, 105.0% for Homi, 143.3% for Learch, 174.3% for AAQC, and 36.7% for SymTuner. These results show that Orbis is complementary to existing symbolic execution techniques despite their different underlying methodologies. For `sqlite`, integrating AAQC with Orbis yielded the highest improvement, achieving approximately 16× higher branch coverage than AAQC alone. Statistical analysis using the Mann-Whitney U test also confirms significant improvements in total branch coverage, with an average p-value of 0.010 across all techniques. The average p-values for each technique are: KLEE (0.009), Homi (0.008), Learch (0.008), AAQC (0.008), and SymTuner (0.015). All p-values are below 0.05.

Interestingly, integrating Orbis with each symbolic execution technique ensures a certain level of branch coverage, even for benchmarks that are inadequately tested when using the technique alone. For certain benchmarks, each technique has limitations that prevent it from fully exploring the program, resulting in less than half of the coverage by the original KLEE (Pure). For example, AAQC alone

Table 4: The total branch coverage achieved by baselines and their combination with Orbis.

Program	KLEE [5]		Homi [10]		Learch [22]		AAQC [42]		SymTuner [8]	
	Pure	Orbis	Pure	Orbis	Pure	Orbis	Pure	Orbis	Pure	Orbis
xorriso	2,474	**10,373**	2,625	**10,626**	3,040	**10,624**	2,139	**10,665**	3,257	**8,818**
sqlite	5,495	**10,266**	4,362	**10,401**	5,773	**10,280**	597	**9,655**	7,460	**10,835**
gawk	3,400	**4,789**	3,618	**4,352**	1,987	**4,207**	3,563	**4,332**	3,747	**3,948**
gcal	2,812	**6,739**	3,195	**6,554**	2,932	**6,612**	1,367	**6,529**	4,012	**6,202**
objdump	599	**1,174**	126	**986**	821	**1,146**	659	**1,188**	1,085	**1,533**
find	1,852	**4,011**	1,531	**3,778**	2,037	**3,767**	2,642	**3,820**	2,766	**4,026**
grep	2,534	**4,068**	2,299	**3,990**	337	**4,162**	2,490	**3,996**	3,482	**4,064**
diff	975	**2,734**	853	**2,731**	929	**2,616**	846	**2,544**	2,046	**2,212**
make	2,226	**3,253**	959	**2,134**	1,063	**3,183**	959	**2,491**	2,957	**3,181**
du	923	**1,364**	1,058	**1,300**	666	**1,325**	588	**1,308**	1,157	**1,385**
patch	1,247	**1,511**	1,043	**1,278**	173	**1,367**	1,271	**1,428**	1,446	**1,580**
objcopy	494	**1,365**	159	**994**	623	**850**	1,304	**1,605**	1,141	**1,208**
ptx	1,428	**2,202**	1,736	**2,221**	1,027	**2,267**	736	**2,251**	2,183	**2,341**
csplit	790	**1,661**	1,584	**1,643**	762	**1,578**	590	**1,652**	1,691	**1,744**
ls	1,355	**2,045**	1,686	**2,011**	860	**2,039**	496	**2,068**	1,803	**1,933**
total	28,605	**57,556**	26,833	**54,998**	23,030	**56,022**	20,247	**55,532**	40,232	**55,011**

Table 5: Number of bugs detected by the baselines and their combinations with Orbis.

Program	KLEE [5]		Homi [10]		Learch [22]		AAQC [42]		SymTuner [8]	
	Pure	Orbis	Pure	Orbis	Pure	Orbis	Pure	Orbis	Pure	Orbis
sqlite	-	**3**	-	**3**	-	**2**	-	**2**	-	**3**
gawk	-	**1**	-	-	-	-	-	**1**	**1**	**1**
gcal	2	**3**	1	**3**	5	**6**	1	**3**	4	**6**
find	-	**1**	**1**	**1**	**1**	**1**	**1**	**1**	**2**	**2**
patch	-	**1**	**1**	**1**	-	**1**	-	**1**	**1**	**1**
total	2	**9**	3	**8**	6	**10**	2	**8**	8	**13**

covered only 597 branches in `sqlite`, just 10.9% of what KLEE alone achieved. However, integrating Orbis with AAQC significantly improved coverage, reaching 9,655 branches, nearly twice as many as KLEE alone. Similar trends are observed in Learch for `grep`, AAQC for `make`, and Homi for `objdump` and `objcopy`. In conclusion, Orbis enables symbolic execution techniques, which primarily cover error-handling functions on certain benchmarks, to explore core functionalities, achieving higher branch coverage.

Across all five baselines, higher option-related branch coverage consistently correlates with higher total branch coverage. Calculating the Pearson correlation coefficient between OB coverage and total branch coverage across all five baselines for each benchmark revealed a strong correlation, averaging 0.902 across 15 benchmarks. Remarkably, the result on `grep` exhibited a correlation of 0.990, indicating an almost perfect linear relationship. Furthermore, all benchmarks showed a correlation of 0.75 or higher, confirming a strong relationship between the two coverage metrics. These results provide evidence that increasing option-related branch coverage, as targeted by Orbis, ultimately leads to a higher total branch coverage, which is the primary goal of symbolic execution.

4.3 Bug-finding Ability

Table 5 demonstrates the effectiveness of integrating Orbis with symbolic execution techniques in bug detection. To derive these results, we re-executed all test-cases generated by the five techniques both with and without Orbis, and collected those that triggered abnormal signals (e.g., segmentation faults) leading to process termination. Bug uniqueness was determined by the location of occurrence, extracted from the "test*.err" files produced by KLEE. We counted unique bugs that were detected at least once within the 24-hour budget across five trials for each technique and benchmark.

On average, Orbis's guidance enabled the five techniques to detect 5.4 additional bugs across five benchmarks (`sqlite`, `gawk`, `gcal`, `find`, and `patch`) compared to their pure versions. Among them, KLEE achieved the largest relative improvements, detecting 4.5 times more bugs than their pure versions. Across all baselines, Orbis not only preserved all bugs detected by the pure techniques but also uncovered additional ones. Notably, techniques with fewer initial detections, such as KLEE, achieved the largest relative improvement (2 → 9), while techniques with higher initial counts, such as SymTuner, reached the highest absolute total (8 → 13). All discovered bugs were reported to the respective developers and were officially confirmed.

Table 6: Orbis's efficiency in option-related branch targeting.

Program	Orbis		Orbis-Extract		Program	Orbis		Orbis-Extract	
	OB	B (Bug)	OB	B (Bug)		OB	B (Bug)	OB	B (Bug)
xorriso	**427.4**	**10,372.8 (0)**	382.8	7,549.4 (0)	make	**65.0**	**3,253.0 (0)**	62.6	2,811.8 (0)
sqlite	**86.2**	**10,266.0 (3)**	65.8	9,612.0 (2)	du	**68.8**	**1,364.2 (0)**	59.6	1,317.4 (0)
gawk	**96.4**	**4,788.6 (1)**	63.0	3,882.2 (0)	patch	**144.8**	**1,511.4 (1)**	123.8	1,317.6 (1)
gcal	**181.4**	**6,739.4 (3)**	170.4	6,425.2 (3)	objcopy	**204.2**	**1,365.2 (0)**	165.2	1,119.6 (0)
objdump	**87.4**	**1,173.6 (0)**	76.2	993.4 (0)	ptx	**52.2**	**2,202.2 (0)**	34.6	2,176.8 (0)
find	**175.0**	**4,011.2 (1)**	147.0	3,196.2 (1)	csplit	**20.6**	**1,661.4 (0)**	20.2	1,534.8 (0)
grep	**153.6**	**4,067.8 (0)**	93.6	3,916.2 (0)	ls	**234.8**	**2,045.4 (0)**	219.8	1,995.2 (0)
diff	**150.2**	**2,733.6 (0)**	139.6	2,323.6 (0)	total	**2,148.0**	**57,555.8 (9)**	1,824.2	50,171.4 (7)

4.4 Effectiveness of Stages

In this section, we evaluate the effectiveness of the three stages in Orbis—Extract, Construct, and Guide—by comparing KLEE+Orbis with variants that exclude each stage on our benchmark suite: Orbis-Extract, RandConst, and RandGuide. We further assess the efficiency of the SMT-solver invocation reduction introduced in the Update stage in KLEE+Orbis.

Efficiency of the Extract Stage. We evaluated the efficiency of the Extract Stage by comparing Orbis, which uses the option-related branches (OBs) extracted in this stage for guidance, with a variant of Orbis that omits this stage (Orbis-Extract). This variant instead employs the total branches (*TotalB*) for guidance. Specifically, it replaced the set *ArgD* constructed at line 4 in Algorithm 2 with the following *ArgD′*:

$$ArgD' = \{(Arg_1, TotalB, CovB), (Arg_2, TotalB, CovB), \cdots, (Arg_n, TotalB, CovB)\}$$

where *TotalB* is the total number of branches in the target program and *CovB* is the number of covered branches when using *Arg*. In other words, this variant targets option arguments by increasing *CovB* rather than OB coverage, and initializes the starting state set using the seed inputs with high branch coverage.

Table 6 shows that targeting option-related branches is effective for exploring core functionality. Compared to Orbis-Extract, Orbis covered 17.8% more option-related branches, achieved 14.7% higher total branch coverage, and detected two additional bugs across 15 programs. Notably, Orbis attained even higher total branch coverage than directly targeting all branches. This is because it forms the initial state set with promising states directly related to targeted option arguments. In practice, Orbis also generated 27.8% more valid test-cases—those that terminate normally without command errors—than Orbis-Extract.

Effectiveness of Construct and Guide Stages. We evaluated the effectiveness of Orbis's two iterative stages—Construct and Guide—by comparing Orbis with four variants: RandConst, FuzzConst, CitConst, and RandGuide. The first three target the Construct stage, while the last targets the Guide stage. Specifically, RandConst replaces the Construct function in Algorithm 2 with a

Table 7: Impact of Construct stage in Orbis.

Program	Orbis		RandConst		FuzzConst		CitConst	
	OB	B (Bug)	OB	B (Bug)	OB	B (Bug)	OB	B (Bug)
xorriso	**427.4**	**10,372.8 (0)**	394.0	8,176.8 (0)	360.4	8,558.0 (0)	171.4	2,396.8 (0)
sqlite	**86.2**	**10,266.0 (3)**	54.0	9,303.4 (1)	74.8	8,508.8 (1)	41.0	4,811.4 (0)
gawk	**96.4**	**4,788.6 (1)**	63.0	3,550.0 (0)	85.8	3,963.2 (1)	59.4	2,874.0 (0)
gcal	**181.4**	**6,739.4 (3)**	165.0	6,436.8 (2)	154.2	6,062.2 (2)	126.8	1,889.8 (1)
objdump	**87.4**	**1,173.6 (0)**	68.0	693.8 (0)	76.8	1,015.6 (0)	67.0	666.6 (0)
find	**175.0**	**4,011.2 (1)**	143.8	3,238.8 (1)	137.2	3,484.4 (1)	149.4	3,196.8 (1)
grep	**153.6**	**4,067.8 (0)**	125.8	3,735.6 (0)	139.8	3,784.0 (0)	94.6	2,488.8 (0)
diff	**150.2**	**2,733.6 (0)**	132.4	1,534.6 (0)	135.2	2,065.4 (0)	86.4	431.4 (0)
make	**65.0**	**3,253.0 (0)**	59.8	2,682.0 (0)	56.2	2,951.0 (0)	49.2	1,920.0 (0)
du	**68.8**	**1,364.2 (0)**	41.0	1,242.2 (0)	62.6	1,283.2 (0)	62.0	1,311.0 (0)
patch	**144.8**	**1,511.4 (1)**	132.6	1,466.8 (1)	120.2	1,159.2 (1)	68.4	950.2 (1)
objcopy	**204.2**	**1,365.2 (0)**	158.8	781.8 (0)	154.2	815.2 (0)	149.2	694.4 (0)
ptx	**52.2**	**2,202.2 (0)**	35.4	1,973.6 (0)	46.8	2,094.0 (0)	45.0	1,693.2 (0)
csplit	**20.6**	**1,661.4 (0)**	18.0	1,449.0 (0)	17.6	1,478.8 (0)	16.2	1,237.2 (0)
ls	**234.8**	**2,045.4 (0)**	210.2	1,983.4 (0)	228.2	1,987.2 (0)	152.6	1,780.0 (0)
total	**2,148.0**	**57,555.8 (9)**	1,801.8	48,248.6 (5)	1,850.0	49,210.2 (6)	1,338.6	28,341.6 (3)

naive version that randomly constructs option arguments from the extracted options. FuzzConst replaces the Construct function with the option-argument construction mechanism of ZigZagFuzz [26], a state-of-the-art program-option fuzzer. Similarly, CitConst replaces the Construct function with the option-argument construction mechanism of HSCA [29], a state-of-the-art combinatorial interaction testing (CIT) technique. Since both ZigZagFuzz and HSCA are publicly available, we used their original implementations without modification. RandGuide modifies the Guide stage by initializing the starting state set S_P using randomly selected inputs from the generated input set (*TotalT*, line 11 of Algorithm 2).

Tables 7 and 8 show that both stages in Orbis are crucial for improving performance. For the Construct stage, Orbis achieved 19.2%, 16.1%, and 60.5% higher option-related branch coverage, 19.3%, 17.0%, and 103.1% higher overall branch coverage, and detected four, three, and six additional unique bugs compared with RandConst, FuzzConst, and CitConst, respectively.

We investigated why Orbis notably outperformed CitConst. The key difference is their objective. As a state-of-the-art CIT technique, CitConst systematically exercises option combinations while minimizing the number of generated option arguments. Consistent with this objective, it selected substantially more options per option argument than Orbis (37.4 on average, 9.4× more). When applied to symbolic execution, however, these large option arguments often resulted in ineffective executions: 68.6% either achieved zero branch coverage or terminated before exhausting the time budget, frequently due to conflicting or ineffective option combinations. Furthermore, CitConst does not prioritize previously exercised combinations, even when they prove effective. Across the 15 programs, its option usage exhibited a lower average Gini coefficient [18] than Orbis (0.072 vs. 0.269), indicating more uniform option usage rather than repeated reuse of effective option arguments. In contrast, Orbis continuously evaluates the effectiveness of option arguments during symbolic execution and repeatedly reuses those that prove effective in improving exploration.

We further compared Orbis+KLEE with KLEE using two simple option-construction strategies. We defined two variants of KLEE: (1) AllConst, KLEE configured with all CLI options extracted by the Extract stage, and (2) OneConst, which repeatedly starts symbolic execution with a single randomly selected option per iteration. Compared with AllConst and OneConst, respectively, Orbis covered 153.4% and 23.4% more branches and detected seven and three additional unique bugs across our benchmark suite. These results indicate that constructing option arguments based on their observed

Table 8: Impact of Guide stage in Orbis.

Program	Orbis		RandGuide		Program	Orbis		RandGuide	
	OB	B (Bug)	OB	B (Bug)		OB	B (Bug)	OB	B (Bug)
xorriso	**427.4**	**10,372.8 (0)**	387.2	6,982.4 (0)	make	**65.0**	**3,253.0 (0)**	57.0	2,781.2 (0)
sqlite	**86.2**	**10,266.0 (3)**	54.4	9,386.0 (2)	du	**68.8**	**1,364.2 (0)**	40.4	1,216.0 (0)
gawk	**96.4**	**4,788.6 (1)**	60.0	3,381.8 (0)	patch	**144.8**	**1,511.4 (1)**	118.6	1,167.8 (1)
gcal	**181.4**	**6,739.4 (3)**	168.0	6,443.4 (2)	objcopy	**204.2**	**1,365.2 (0)**	155.6	726.6 (0)
objdump	**87.4**	**1,173.6 (0)**	59.6	613.4 (0)	ptx	**52.2**	**2,202.2 (0)**	38.8	1,946.6 (0)
find	**175.0**	**4,011.2 (1)**	162.8	3,647.0 (1)	csplit	**20.6**	**1,661.4 (0)**	20.2	1,484.8 (0)
grep	**153.6**	**4,067.8 (0)**	95.4	3,640.0 (0)	ls	**234.8**	**2,045.4 (0)**	216.8	1,963.2 (0)
diff	**150.2**	**2,733.6 (0)**	140.0	2,004.8 (0)	total	**2,148.0**	**57,555.8 (9)**	1,774.8	47,385.0 (6)

Table 9: Evaluation of Orbis's solver-skipping approach.

Program	Orbis			CallAll			SMT Skip	Hit Ratio	Saved Time
	OB	B	Bug	OB	B	Bug	(# calls)	(%)	(sec)
xorriso	**427.4**	**10,372.8**	-	396.6	7,847.4	-	1,362,355	11.4%	1,789
sqlite	**86.2**	**10,266.0**	**3**	62.0	9,518.8	2	86,479	44.1%	27,278
gawk	**96.4**	**4,788.6**	**1**	67.2	3,384.4	-	462,355	19.0%	10,422
gcal	**181.4**	**6,739.4**	**3**	168.6	6,577.6	**3**	435,430	15.2%	11,557
objdump	**87.4**	**1,173.6**	-	77.4	1,011.6	-	1,333,503	35.7%	27,190
find	**175.0**	**4,011.2**	**1**	163.8	3,705.4	**1**	160,856	6.9%	4,412
grep	**153.6**	**4,067.8**	-	114.2	3,655.2	-	477,565	18.1%	13,750
diff	**150.2**	**2,733.6**	-	127.4	1,680.8	-	695,607	16.3%	11,255
make	**65.0**	**3,253.0**	-	59.8	2,649.2	-	574,861	19.7%	9,945
du	**68.8**	**1,364.2**	-	52.0	1,316.0	-	523,461	14.9%	12,194
patch	**144.8**	**1,511.4**	**1**	129.2	1,388.0	**1**	1,218,105	18.0%	12,410
objcopy	**204.2**	**1,365.2**	-	159.0	1,127.4	-	975,411	18.7%	14,629
ptx	**52.2**	**2,202.2**	-	34.8	1,800.0	-	389,418	15.4%	12,323
csplit	**20.6**	**1,661.4**	-	20.4	1,511.2	-	52,624	8.3%	6,825
ls	**234.8**	**2,045.4**	-	219.8	1,669.4	-	154,341	17.7%	12,126
total	**2,148.0**	**57,555.8**	**9**	1,852.2	48,842.4	7	3,840,978	16.3%	188,104

effectiveness is substantially more effective than either enabling all extracted CLI options simultaneously or using individual options.

For the Guide stage, Orbis achieved 21.0% higher OB coverage, resulting in a 21.5% increase in total branch coverage and 50% more bugs detected compared to RandGuide. Overall, these results indicate the clear benefit of Orbis, which carefully constructs option arguments in the Construct stage and efficiently initializes promising states in the Guide stage.

Efficiency of Skipping Solver Calls. We also evaluated how Orbis's efficiency is improved by the SMT-solver call reduction method introduced in the Update stage. Specifically, we compared Orbis with a variant (CallAll) that invokes the solver for every halt statement. Table 9 shows that Orbis's approach of generating test-cases without invoking the solver has a substantial impact on performance. Compared to CallAll, Orbis achieved 16.0% higher option-related branch coverage, 17.8% higher total branch coverage, and detected one additional bug each in `sqlite` and `gawk`. This improvement arises from reducing the constraint-solving overhead of symbolic execution. Empirically, over 360 hours of testing (24 hours × 15 benchmarks), Orbis reduced solving cost for approximately 8.9 million statements, corresponding to 16.8% of all halt statements. In terms of time, this reduction saved about 188,000 seconds, enabling an average of three additional hours of exploration per benchmark. Notably, for `ls` and `grep`, whose behaviors heavily depend on directory structures and file contents, Orbis skipped more than 150K and 470K SMT calls, respectively, while still improving branch coverage (`ls`: 1,669→2,045, `grep`: 3,655→4,067). These results demonstrate that Orbis can further optimize the guidance of symbolic execution techniques toward maximizing OB coverage.

4.5 Generality

To evaluate the generality of Orbis, we integrated it into concolic testing [20, 38], a major variation of dynamic symbolic execution. Specifically, we incorporated Orbis with CREST [4], a widely used concolic testing tool, and Chameleon [9], a state-of-the-art search strategy built on top of CREST. For this evaluation, we used the four

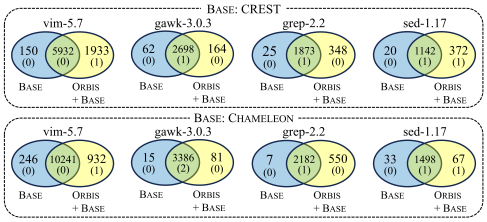

Figure 3: The efficiency of branch coverage and bug finding ability by concolic testing tools with and without Orbis.

largest benchmark programs previously employed in Chameleon's evaluation: vim-5.7, gawk-3.0.3, grep-2.2, and sed-1.17. These programs contain 27, 16, 29, and 4 options, respectively. Orbis aims to guide concolic testing techniques to quickly reach option-related branches: `vim` (1,548 OBs), `gawk` (293 OBs), `grep` (278 OBs), and `sed` (14 OBs). Integrating Orbis into a concolic testing engine corresponds to replacing the Execute procedure (Algorithm 1) with a concolic testing procedure [4, 9]. The main difference is that Algorithm 1 takes as input a promising state set S_P initialized by multiple seed inputs, whereas these two concolic testing tools directly utilize the seed inputs without explicit state initialization. To address this, we modified the output of the Guide stage in Algorithm 2 to provide the selected seed inputs instead of a promising state set. For the evaluation, we allocated a total testing budget of 24 hours per program, reporting the average results of five trials.

Figure 3 shows that Orbis improves total branch coverage and bug-finding capability in concolic testing tools. On average, integrating Orbis increased total branch coverage by 21.5% for CREST and 7.6% for Chameleon. For uniquely covered branches, Orbis covered 10.9x and 5.4x more branches than the corresponding tools without Orbis. Orbis also enabled CREST and Chameleon to detect five and six bugs, respectively. The integrated versions detected all bugs found by their pure counterparts and additionally discovered two more bugs in `vim` and `sed`. These results demonstrate Orbis's generality and its ability to consistently improve branch coverage and bug detection across diverse symbolic executors.

4.6 Case Study

To illustrate how Orbis guides symbolic execution to achieve higher total branch coverage, we conducted a case study on the `diff` program. Particularly, we analyzed an effective test-case ("diff -p -0") that covers 471 branches in a single execution, accounting for 48.3% of the total coverage achieved by KLEE within a 24-hour budget. We then compared the average time required by KLEE with and without Orbis's guidance to generate this test-case. The simplified code snippet below illustrates how Orbis efficiently generates it:

```
1  int opt0 = -1; char style = "+";
2  opt0 = getopt_long(argc, argv); /* Function for parsing options */
3  /* ... Other branch conditions ... */
4  switch (opt0) {
5      case "a": /* Handle option "-a" */ break;
6      /* ... Define other 48 case statements ... */
7      case "p":
8          if (style == "+") { /* Code unrelated to options */ }
9          /* ... Other conditions ... */
10         opt1 = getopt_long(argc, argv);
11         if (opt1 == "0") {/* Handle option "-p -0" */}
12 }
```

Table 10: Comparing Orbis+KLEE with option-configuring techniques in fuzzing. Format: unique branches(bugs)

Program	Orbis	Both	ZigZagFuzz	Orbis	Both	OSmart	Orbis	Both	CarpetFuzz
xorriso	3,297(0)	7,076(0)	3,474(0)	8,505(0)	1,868(0)	874(0)	8,140(0)	2,233(0)	147(0)
sqlite	1,896(2)	8,370(1)	2,544(0)	7,701(2)	2,565(1)	772(0)	6,433(2)	3,833(1)	98(0)
gawk	1,045(1)	3,744(0)	682(0)	1,785(1)	3,003(0)	227(0)	2,229(1)	2,559(0)	81(0)
gcal	356(1)	6,384(2)	2,256(0)	5,237(3)	1,502(0)	50(0)	4,919(2)	1,820(1)	0(0)
objdump	300(0)	874(0)	5,968(1)	615(0)	559(0)	1,921(1)	329(0)	845(0)	3,538(1)
find	1,682(0)	2,329(1)	352(0)	2,779(1)	1,232(0)	189(0)	2,522(1)	1,489(0)	40(0)
grep	698(0)	3,369(0)	430(0)	1,457(0)	2,611(0)	70(0)	1,905(0)	2,162(0)	43(0)
diff	672(0)	2,062(0)	892(0)	1,909(0)	824(0)	48(0)	1,945(0)	789(0)	52(0)
make	1,320(0)	1,933(0)	48(0)	2,009(0)	1,244(0)	29(0)	1,778(0)	1,475(0)	102(0)
du	232(0)	1,132(0)	95(0)	547(0)	817(0)	51(0)	644(0)	720(0)	51(0)
patch	457(0)	1,055(1)	968(0)	791(0)	720(1)	660(0)	829(0)	682(1)	420(0)
objcopy	762(0)	603(0)	1,589(0)	801(0)	564(0)	1,476(0)	877(0)	488(0)	1,516(0)
ptx	265(0)	1,938(0)	122(0)	1,008(0)	1,194(0)	46(0)	1,065(0)	1,137(0)	80(0)
csplit	292(0)	1,369(0)	625(0)	1,228(0)	433(0)	36(0)	1,125(0)	536(0)	26(0)
ls	121(0)	1,925(0)	148(0)	1,037(0)	1,008(0)	56(0)	1,566(0)	480(0)	17(0)
total	13,395(4)	44,163(5)	20,193(1)	37,419(7)	20,144(2)	6,505(1)	36,306(6)	21,248(3)	6,211(1)

When using KLEE alone, numerous states must be traversed before reaching the states corresponding to the option "-p" (line 7), since the switch statement contains 50 cases. In contrast, Orbis-guided KLEE leveraged "-p" both as an option argument and as an initial promising state set, steering exploration directly toward relevant states—those including the branch condition at line 7—rather than starting from line 1. Empirically, KLEE with Orbis generated the test-case "diff -p -0" in just 7.1 seconds. Here, "-p" is an option argument constructed by the Construct stage, whereas "-0" is a symbolic argument generated by KLEE through SMT solving. In contrast, KLEE alone required 9,012.7 seconds to generate the entire argument "-p -0" symbolically—approximately 1,269× longer. Furthermore, thanks to the SMT-solving reduction method introduced in the Update stage, Orbis reduced solver calls and generated this test-case 7.91x faster than the Orbis variant without SMT skipping.

4.7 Comparison with Fuzzing Techniques

We compared KLEE+Orbis with three state-of-the-art fuzzing techniques in testing program options—ZigZagFuzz [26], OSmart [45], and CarpetFuzz [44]—in branch coverage and bug-detection. For this comparison, we used the publicly available implementations without modification. However, unlike Orbis, which operates fully automatically, these fuzzers require manual intervention to function effectively, including specifying available options and their valid argument values (e.g., -e "abc|cd") and preparing an initial seed input for each program. For a fair comparison, we provided the fuzzers with all information obtained from the Extract stage—i.e., the available options and their valid argument values. In addition, because these fuzzers require an initial seed input, we manually collected valid test cases from each target program's documentation and selected as the initial seed the one covering the largest number of OBs. Without such manual intervention, their performance degrades substantially. Therefore, we provided the necessary information for each program and evaluated all fuzzers on our benchmark suite, running each fuzzer for 24 hours, repeated five times per benchmark, and reported the average results.

Table 10 shows that Orbis explores a large number of branches that remain unreachable by state-of-the-art fuzzers and outperforms them in terms of bug-finding capability. Across 15 benchmarks, Orbis exclusively covered 13,395, 37,419, and 36,306 branches that ZigZagFuzz, OSmart, and CarpetFuzz failed to reach, respectively. In particular, among the 13,395 branches not covered by

ZigZagFuzz, 92.1% remained unreachable even when its option-configuration method replaced Orbis's Construct stage (FuzzConst in Table 7), highlighting that these branches can only be reached with option configurations tailored to symbolic execution. In contrast, these fuzzers exclusively covered 20,193, 6,505, and 6,211 branches, primarily because they leverage diverse inputs generated by mutating manually provided seeds, while Orbis operates without such manual seeds and relies on symbolic inputs. In terms of bug detection, Orbis detected four, seven, and six additional bugs compared to ZigZagFuzz, OSmart, and CarpetFuzz, respectively, while missing only one bug in `objdump`.

4.8 Threats to Validity

(1) In the Extract stage, Orbis identifies options solely from the program's help output and thus may miss hidden options not listed there. However, our manual inspection shows that fewer than one hidden option exists per benchmark program on average (0.7 per program). Moreover, even when manually incorporated, these options have negligible impacts, improving branch coverage by only about 1%. Therefore, we adopt a cost-effective strategy that extracts options exclusively from the help output. (2) Orbis identifies option-related branches (OBs) from (variable, value) dependencies in execution-trace differences. It does not explicitly consider indirect control dependencies that arise solely through execution reachability. For example, one option-controlled branch may determine whether a later, unrelated branch is reached, even though no option-specific (variable, value) dependency exists between them. While our results indicate that the extracted OBs effectively guide symbolic execution, incorporating more precise control-dependence analysis could further improve performance. (3) We integrated Orbis into two dynamic symbolic executors, KLEE and CREST, significantly improving their performance. However, they may not generalize to other symbolic execution engines [12, 21].

5 Related Work

Boosting Symbolic Execution. To our knowledge, Orbis is the first technique to improve symbolic execution by guiding it to reach option-related branches. Existing approaches improve symbolic execution through various methodologies [4, 8, 10, 11, 22–24, 34, 39, 41, 42, 46, 49]. Search heuristics [4, 22, 41, 49] prioritize states to maximize code coverage and trigger bugs based on predefined criteria, such as states containing less exercised subpaths [27] or states with the shortest distance to uncovered code regions [4]. Second, state-pruning strategies [3, 10, 11, 23] improve symbolic execution efficiency by eliminating redundant states using unique criteria, such as states executing the same code blocks as previously explored ones [3], states that are likely to generate invalid inputs [46], and states that fail to reach annotated bug locations [23]. Third, constraint-solving methods [24, 34, 39, 42] focus on reducing the computational cost of solving specific constraint types, including array constraints [34] and floating-point constraints [28]. Lastly, SymTuner [8] automatically tunes the external parameters of a symbolic executor (e.g., KLEE) from a manually specified parameter space. In contrast, Orbis improves symbolic execution by constructing option arguments for the target program and guiding exploration toward OBs. Since Orbis is a complementary technique, integrating it with existing approaches improves their performance.

Software Testing with Option Arguments. Our work relates to software testing techniques that construct valid option arguments for target programs [22, 24–26, 40, 44, 45, 50], but differs in how option arguments are defined and utilized. Several symbolic execution techniques [22, 24] manually define option arguments for each program and concatenate them with generated inputs to enhance coverage. Some grey-box fuzzing techniques [25, 26, 40, 44, 45, 50] attempt to test a wide range of option arguments by mutating options extracted from the program's manual pages and source code. In contrast, Orbis automatically constructs option arguments while interacting with a symbolic executor, requiring no prior knowledge. It further leverages these arguments to initialize the symbolic executor states and reduce SMT-solver invocations, maximizing option-related coverage.

Software Testing for Specific Code Regions. Our approach aligns with techniques designed to reach specific code regions within a program [2, 14, 16, 31, 33, 35, 36, 47, 48]. However, Orbis differs in both the target regions and the methodology. For instance, directed symbolic execution [31] prioritizes code lines associated with bugs (e.g., failing assertions) and quickly reaches them using specialized search heuristics. Oasis [14] and KATCH [33] focus on covering newly added patch codes by leveraging heuristics based on static and dynamic analysis. AFLGo [2] prioritizes user-defined target regions using seeds with the shortest distance, while AFLSmart++ [35] targets hard-to-reach input chunks by delicately mutating the seed that can reach those chunks. Unlike these approaches, Orbis is the first symbolic execution technique that specifically targets option-related branches by automatically constructing option arguments and initializing the corresponding symbolic states.

6 Conclusion

Recent advancements in symbolic execution have improved its ability to uncover hard-to-reach bugs and code regions. However, these techniques still struggle to explore option-related code regions, which correspond to core program functionalities. Orbis addresses this limitation by automatically constructing effective option arguments and optimizing initial states, while reducing unnecessary SMT solver invocations. Experimental results show that integrating Orbis with five advanced techniques yields significant gains in branch coverage and bug detection beyond existing approaches.

Acknowledgments

This work was supported by the National Research Foundation of Korea(NRF) grant funded by the Korea government(MSIT) (RS-2026-25497428, RS-2026-25521371). This work was also supported by the Institute of Information & Communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) (2022-0-00688, 2024-00337703, 2024-00398745, No.RS-2024-00438686, Development of software reliability improvement technology through identification of abnormal open sources and automatic application of DevSecOps).

Data Availability. Orbis is accessible at: https://doi.org/10.5281/zenodo.21767392.

References

[1] Dirk Beyer and Thomas Lemberger. 2018. CPA-SymExec: efficient symbolic execution in CPAchecker. In *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering*. 900–903. doi:10.1145/3238147.3240478

[2] Marcel Böhme, Van-Thuan Pham, Manh-Dung Nguyen, and Abhik Roychoudhury. 2017. Directed greybox fuzzing. In *Proceedings of the 2017 ACM SIGSAC conference on computer and communications security*. 2329–2344. doi:10.1145/3133956.3134020

[3] Peter Boonstoppel, Cristian Cadar, and Dawson Engler. 2008. RWset: Attacking path explosion in constraint-based test generation. In *International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS '08)*. 351–366. doi:10.1007/978-3-540-78800-3_27

[4] Jacob Burnim and Koushik Sen. 2008. Heuristics for Scalable Dynamic Test Generation. In *Proceedings of 23rd IEEE/ACM International Conference on Automated Software Engineering (ASE '08)*. 443–446. doi:10.1109/ASE.2008.69

[5] Cristian Cadar, Daniel Dunbar, and Dawson Engler. 2008. KLEE: Unassisted and Automatic Generation of High-coverage Tests for Complex Systems Programs. In *Proceedings of the 8th USENIX Conference on Operating Systems Design and Implementation (OSDI '08)*. 209–224.

[6] Cristian Cadar and Dawson Engler. 2005. Execution Generated Test Cases: How to Make Systems Code Crash Itself. In *Proceedings of the 12th International Conference on Model Checking Software (SPIN'05)*. 2–23. doi:10.1007/11537328_2

[7] Sooyoung Cha, Seongjoon Hong, Jiseong Bak, Jingyoung Kim, Junhee Lee, and Hakjoo Oh. 2022. Enhancing Dynamic Symbolic Execution by Automatically Learning Search Heuristics. *IEEE Transactions on Software Engineering* (2022), 3640–3663. doi:10.1109/TSE.2021.3101870

[8] Sooyoung Cha, Myungho Lee, Seokhyun Lee, and Hakjoo Oh. 2022. SymTuner: Maximizing the Power of Symbolic Execution by Adaptively Tuning External Parameters. In *Proceedings of the 44th International Conference on Software Engineering (ICSE '22)*. 2068–2079. doi:10.1145/3510003.3510185

[9] Sooyoung Cha and Hakjoo Oh. 2019. Concolic Testing with Adaptively Changing Search Heuristics. In *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '19)*. 235–245. doi:10.1145/3338906.3338964

[10] Sooyoung Cha and Hakjoo Oh. 2020. Making Symbolic Execution Promising by Learning Aggressive State-Pruning Strategy. In *The 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '20)*. doi:10.1145/3368089.3409755

[11] Ying-Shen Chen, Wei-Ning Chen, Che-Yu Wu, Hsu-Chun Hsiao, and Shih-Kun Huang. 2018. Dynamic Path Pruning in Symbolic Execution. In *2018 IEEE Conference on Dependable and Secure Computing (DSC)*. IEEE, 1–8. doi:10.1109/desec.2018.8625128

[12] Vitaly Chipounov, Volodymyr Kuznetsov, and George Candea. 2011. S2E: A platform for in-vivo multi-path analysis of software systems. *Acm Sigplan Notices* 46, 3 (2011), 265–278. doi:10.1145/1961295.1950396

[13] CREST. A concolic test generation tool for C. 2008. https://github.com/jburnim/crest.

[14] Olivier Crameri, Rekha Bachwani, Tim Brecht, Ricardo Bianchini, Dejan Kostic, and Willy Zwaenepoel. 2009. *Oasis: Concolic Execution Driven by Test Suites and Code Modifications*. Technical Report. Technical Report LABOS-REPORT-2009-002, EPFL.

[15] Leonardo De Moura and Nikolaj Bjørner. 2008. Z3: An efficient SMT solver. In *International conference on Tools and Algorithms for the Construction and Analysis of Systems*. Springer, 337–340. doi:10.1007/978-3-540-78800-3_24

[16] Peter Dinges and Gul Agha. 2014. Targeted test input generation using symbolic-concrete backward execution. In *Proceedings of the 29th ACM/IEEE International Conference on Automated Software Engineering (ASE '14)*. 31–36. doi:10.1145/2642937.2642951

[17] Vijay Ganesh and David L Dill. 2007. A decision procedure for bit-vectors and arrays. In *International conference on computer aided verification*. Springer, 519–531. doi:10.1007/978-3-540-73368-3_52

[18] Corrado Gini. 1921. Measurement of inequality of incomes. *The economic journal* 31, 121 (1921), 124–125. doi:10.2307/2223319

[19] GNU Compiler Collection. 2025. *GCC Gcov Manual*. https://gcc.gnu.org/onlinedocs/gcc/Gcov.html

[20] Patrice Godefroid, Nils Klarlund, and Koushik Sen. 2005. DART: Directed Automated Random Testing. In *Proceedings of the 2005 ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '05)*. 213–223. doi:10.1007/978-3-642-19237-1_4

[21] Patrice Godefroid, Michael Y. Levin, and David Molnar. 2012. SAGE: Whitebox Fuzzing for Security Testing: SAGE Has Had a Remarkable Impact at Microsoft. *Queue* (2012), 20–27. doi:10.1145/2093548.2093564

[22] Jingxuan He, Gishor Sivanrupan, Petar Tsankov, and Martin Vechev. 2021. Learning to Explore Paths for Symbolic Execution. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security (CCS '21)*. 2526–2540. doi:10.1145/3460120.3484813

[23] Joxan Jaffar, Vijayaraghavan Murali, and Jorge A. Navas. 2013. Boosting Concolic Testing via Interpolation. In *Proceedings of the 9th Joint Meeting on Foundations of Software Engineering (ESEC/FSE '13)*. 48–58. doi:10.1145/2491411.2491425

[24] Timotej Kapus, Frank Busse, and Cristian Cadar. 2020. Pending constraints in symbolic execution for better exploration and seeding. In *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering*. 115–126. doi:10.1145/3324884.3416589

[25] Ahcheong Lee, Irfan Ariq, Yunho Kim, and Moonzoo Kim. 2022. POWER: Program option-aware fuzzer for high bug detection ability. In *2022 IEEE Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 220–231. doi:10.1109/icst53961.2022.00032

[26] Ahcheong Lee, Youngseok Choi, Shin Hong, Yunho Kim, Kyutae Cho, and Moonzoo Kim. 2025. ZigZagFuzz: Interleaved Fuzzing of Program Options and Files. *ACM Transactions on Software Engineering and Methodology* 34, 2 (2025), 1–31. doi:10.1145/3697014

[27] You Li, Zhendong Su, Linzhang Wang, and Xuandong Li. 2013. Steering Symbolic Execution to Less Traveled Paths. In *Proceedings of the 2013 ACM SIGPLAN International Conference on Object Oriented Programming Systems Languages, and Applications (OOPSLA '13)*. 19–32. doi:10.1145/2544173.2509553

[28] Daniel Liew, Daniel Schemmel, Cristian Cadar, Alastair F Donaldson, Rafael Zahl, and Klaus Wehrle. 2017. Floating-point symbolic execution: a case study in n-version programming. In *2017 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 601–612. doi:10.1109/ase.2017.8115670

[29] Chuan Luo, Shuangyu Lyu, Wei Wu, Hongyu Zhang, Dianhui Chu, and Chunming Hu. 2025. Towards High-Strength Combinatorial Interaction Testing for Highly Configurable Software Systems. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, 650–650. doi:10.1109/ICSE55347.2025.00113

[30] Sicheng Luo, Hui Xu, Yanxiang Bi, Xin Wang, and Yangfan Zhou. 2021. Boosting symbolic execution via constraint solving time prediction (experience paper). In *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis*. 336–347. doi:10.1145/3460319.3464813

[31] Kin-Keung Ma, Khoo Yit Phang, Jeffrey S Foster, and Michael Hicks. 2011. Directed symbolic execution. In *Static Analysis: 18th International Symposium, SAS 2011, Venice, Italy, September 14-16, 2011. Proceedings 18*. 95–111. doi:10.1007/978-3-642-23702-7_11

[32] Henry B Mann and Donald R Whitney. 1947. On a test of whether one of two random variables is stochastically larger than the other. *The annals of mathematical statistics* (1947), 50–60. doi:10.1214/aoms/1177730491

[33] Paul Dan Marinescu and Cristian Cadar. 2013. KATCH: high-coverage testing of software patches. In *Proceedings of the 2013 9th Joint Meeting on Foundations of Software Engineering (ESEC/FSE 2013)*. 235–245. doi:10.1145/2491411.2491438

[34] David M Perry, Andrea Mattavelli, Xiangyu Zhang, and Cristian Cadar. 2017. Accelerating array constraints in symbolic execution. In *Proceedings of the 26th ACM SIGSOFT International Symposium on Software Testing and Analysis*. 68–78. doi:10.1145/3092703.3092728

[35] Van-Thuan Pham. 2023. AFLSmart++: smarter greybox fuzzing. In *2023 IEEE/ACM International Workshop on Search-Based and Fuzz Testing (SBFT)*. IEEE, 76–79. doi:10.1109/sbft59156.2023.00023

[36] Van-Thuan Pham, Marcel Böhme, Andrew E Santosa, Alexandru Răzvan Căciulescu, and Abhik Roychoudhury. 2019. Smart greybox fuzzing. *IEEE Transactions on Software Engineering* 47, 9 (2019), 1980–1997. doi:10.1109/tse.2019.2941681

[37] Heinz Riener, Finn Haedicke, Stefan Frehse, Mathias Soeken, Daniel Große, Rolf Drechsler, and Goerschwin Fey. 2017. metaSMT: focus on your application and not on solver integration. *International Journal on Software Tools for Technology Transfer* 19, 5 (2017), 605–621. doi:10.1007/s10009-016-0426-1

[38] Koushik Sen, Darko Marinov, and Gul Agha. 2005. CUTE: A Concolic Unit Testing Engine for C. In *Proceedings of the 10th European Software Engineering Conference Held Jointly with 13th ACM SIGSOFT International Symposium on Foundations of Software Engineering (ESEC/FSE '05)*. 263–272. doi:10.1145/1095430.1081750

[39] Ziqi Shuai, Zhenbang Chen, Kelin Ma, Kunlin Liu, Yufeng Zhang, Jun Sun, and Ji Wang. 2024. Partial Solution Based Constraint Solving Cache in Symbolic Execution. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 2493–2514. doi:10.1145/3660817

[40] Suhwan Song, Chengyu Song, Yeongjin Jang, and Byoungyoung Lee. 2020. CrFuzz: Fuzzing multi-purpose programs through input validation. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 690–700. doi:10.1145/3368089.3409769

[41] Yue Sun, Guowei Yang, Shichao Lv, Zhi Li, and Limin Sun. 2024. Concrete Constraint Guided Symbolic Execution. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–12. doi:10.1145/3597503.3639078

[42] David Trabish, Shachar Itzhaky, and Noam Rinetzky. 2021. Address-aware query caching for symbolic execution. In *2021 14th IEEE Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 116–126. doi:10.1109/icst49551.2021.00023

[43] David Trabish, Andrea Mattavelli, Noam Rinetzky, and Cristian Cadar. 2018. Chopped symbolic execution. In *Proceedings of the 40th International Conference on Software Engineering*. 350–360. doi:10.1145/3180155.3180251

[44] Dawei Wang, Ying Li, Zhiyu Zhang, and Kai Chen. 2023. {CarpetFuzz}: Automatic program option constraint extraction from documentation for fuzzing. In *32nd USENIX Security Symposium (USENIX Security 23)*. 1919–1936.

[45] Kelin Wang, Mengda Chen, Liang He, Purui Su, Yan Cai, Jiongyi Chen, Bin Zhang, Chao Feng, and Chaojing Tang. 2024. OSmart: Whitebox Program Option Fuzzing. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*. 705–719. doi:10.1145/3658644.3690228

[46] E. Wong, L. Zhang, S. Wang, T. Liu, and L. Tan. 2015. DASE: Document-Assisted Symbolic Execution for Improving Automated Software Testing. In *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering (ICSE '15)*. 620–631. doi:10.1109/icse.2015.78

[47] Jie Wu, Chengyu Zhang, and Geguang Pu. 2020. Reinforcement learning guided symbolic execution. In *2020 IEEE 27th International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 662–663. doi:10.1109/saner48275.2020.9054815

[48] Guowei Yang, Suzette Person, Neha Rungta, and Sarfraz Khurshid. 2014. Directed incremental symbolic execution. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 24, 1 (2014), 1–42. doi:10.1145/2345156.1993558

[49] Jaehan Yoon and Sooyoung Cha. 2024. FeatMaker: Automated Feature Engineering for Search Strategy of Symbolic Execution. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 2447–2468. doi:10.1145/3660815

[50] Zenong Zhang, George Klees, Eric Wang, Michael Hicks, and Shiyi Wei. 2023. Fuzzing configurations of program options. *ACM Transactions on Software Engineering and Methodology* 32, 2 (2023), 1–21. doi:10.14722/fuzzing.2022.23008

Received 2026-03-26; accepted 2026-06-18